

Create Database

Database Name:

bankdb

Collection Name:

Customers

Collection Name:

Transactions

Final structure:

bankdb

|
+---- customers

|
+---- transactions

The screenshot displays the MongoDB Cloud Explorer interface. The left sidebar shows the 'Data Explorer' with a tree view of clusters. The main panel shows the 'customers' collection in the 'bank_DB' database. The 'Documents' tab is active, displaying a list of 8 documents. Each document contains fields: '_id' (ObjectId), 'customer_id' (Int32), 'name' (String), and 'city' (String). The documents are numbered 1 through 8, corresponding to the 'customer_id' values 101 through 108.

	_id	customer_id	name	city
1	ObjectId('6a8964e3c4c890d...')	101	"Ravi Kumar"	"Hyderabad"
2	ObjectId('6a8965bfc4c890d...')	102	"Priya Sharma"	"Vijayawada"
3	ObjectId('6a8965bfc4c890d...')	103	"Arjun Reddy"	"Hyderabad"
4	ObjectId('6a8965bfc4c890d...')	104	"Sneha Rao"	"Warangal"
5	ObjectId('6a8965bfc4c890d...')	105	"Kiran Kumar"	"Guntur"
6	ObjectId('6a8965bfc4c890d...')	106	"Anjali Singh"	"Hyderabad"
7	ObjectId('6a8965bfc4c890d...')	107	"Rahul Verma"	"Bengaluru"
8	ObjectId('6a8965bfc4c890d...')	108	"Meena Devi"	"Chennai"

PART 3: INSERT CUSTOMER DATA

Open:

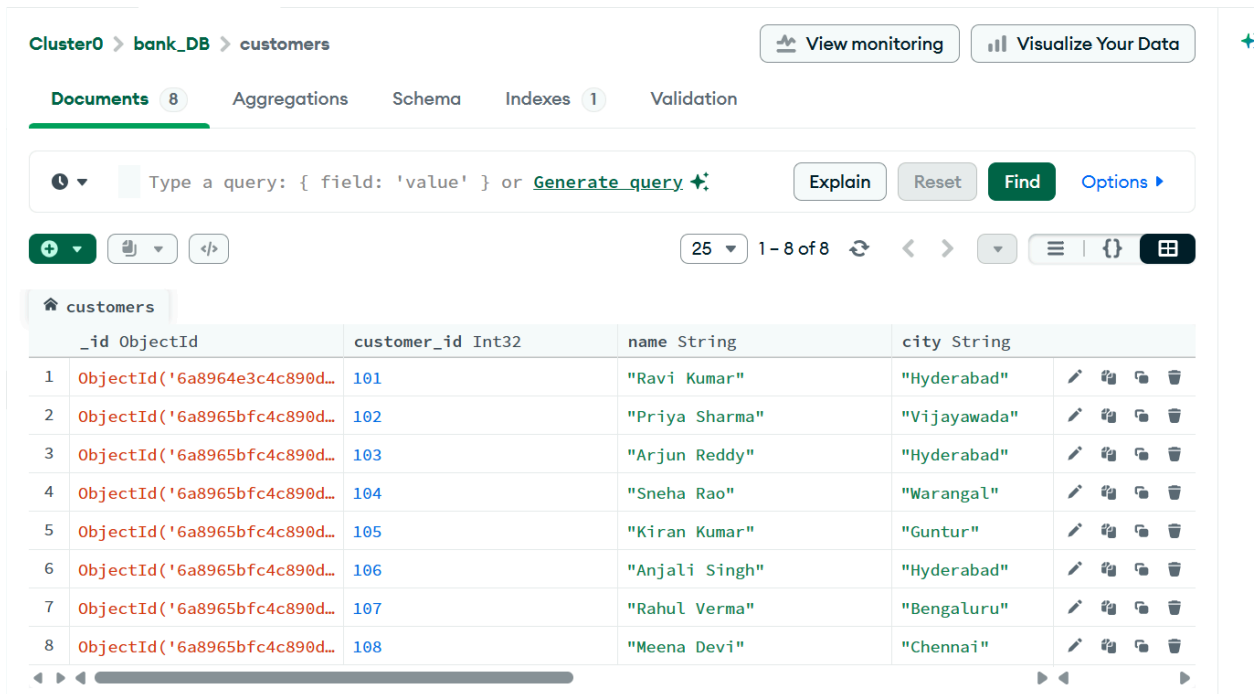
```
bankdb
|
+---- customers
|
+---- Documents
```

Click:

Add Data

```
|
+---- Insert Document
```

Insert the following documents one by one.



The screenshot shows the MongoDB Compass interface for the 'bank_DB' database, specifically the 'customers' collection. The 'Documents' tab is active, showing 8 documents. The interface includes a query bar at the top with a placeholder 'Type a query: { field: 'value' } or [Generate query](#)'. Below the query bar, there are buttons for 'Explain', 'Reset', 'Find', and 'Options'. The document list is displayed in a table format with columns: _id, customer_id, name, and city. Each document is numbered 1 through 8. The table also includes action icons (edit, delete, etc.) for each document.

	_id	customer_id	name	city
1	ObjectId('6a8964e3c4c890d...')	101	"Ravi Kumar"	"Hyderabad"
2	ObjectId('6a8965bfc4c890d...')	102	"Priya Sharma"	"Vijayawada"
3	ObjectId('6a8965bfc4c890d...')	103	"Arjun Reddy"	"Hyderabad"
4	ObjectId('6a8965bfc4c890d...')	104	"Sneha Rao"	"Warangal"
5	ObjectId('6a8965bfc4c890d...')	105	"Kiran Kumar"	"Guntur"
6	ObjectId('6a8965bfc4c890d...')	106	"Anjali Singh"	"Hyderabad"
7	ObjectId('6a8965bfc4c890d...')	107	"Rahul Verma"	"Bengaluru"
8	ObjectId('6a8965bfc4c890d...')	108	"Meena Devi"	"Chennai"

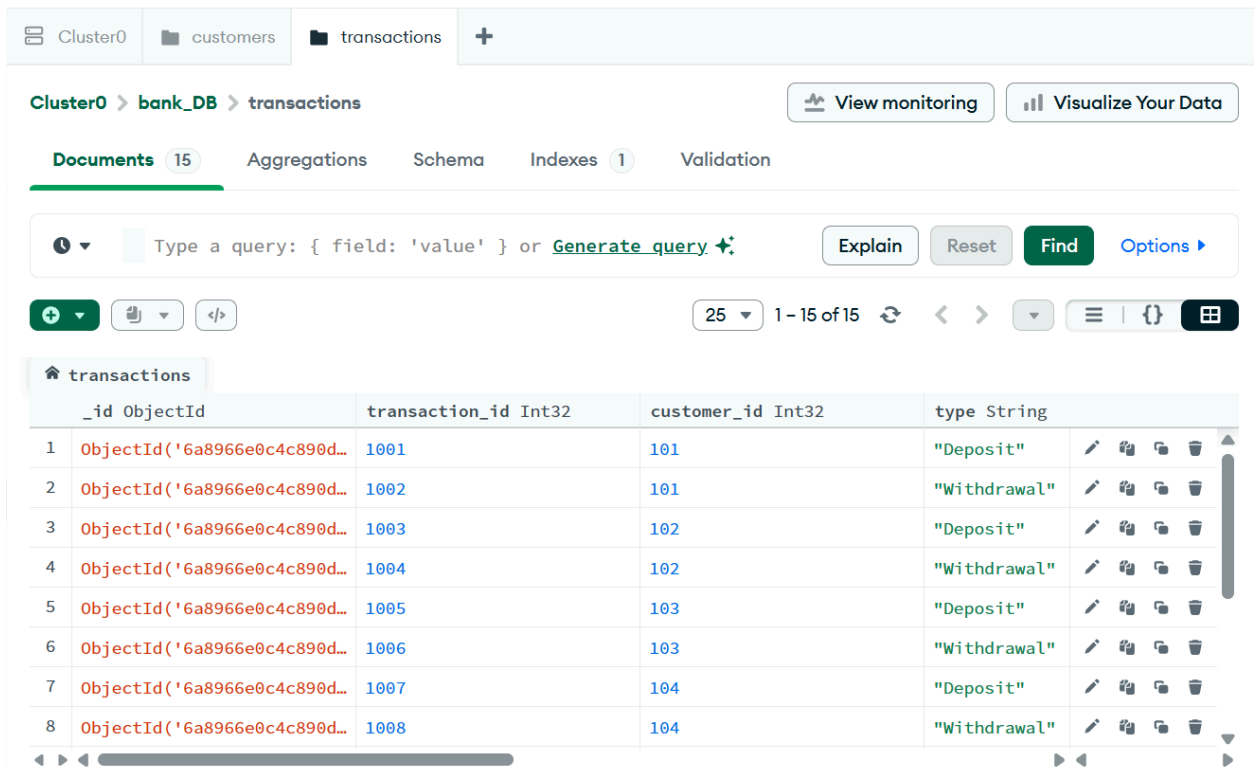
PART 4: INSERT TRANSACTION DATA

Open:

```
bankdb
|
+---- transactions
```

Click:

```
Add Data
|
+---- Insert Document
```



The screenshot shows the MongoDB Compass interface. At the top, the breadcrumb navigation is 'Cluster0 > bank_DB > transactions'. Below this, there are tabs for 'Documents' (15), 'Aggregations', 'Schema', 'Indexes' (1), and 'Validation'. The 'Documents' tab is selected. A search bar is present with the placeholder text 'Type a query: { field: 'value' } or [Generate query](#)'. Below the search bar, there are buttons for 'Explain', 'Reset', 'Find', and 'Options'. The main area displays a table of documents from the 'transactions' collection. The table has columns: '_id', 'transaction_id', 'customer_id', and 'type'. There are 8 documents listed, alternating between 'Deposit' and 'Withdrawal' types. Each document has a unique '_id' and a corresponding 'transaction_id' and 'customer_id'.

	_id	transaction_id	customer_id	type
1	ObjectId('6a8966e0c4c890d...	1001	101	"Deposit"
2	ObjectId('6a8966e0c4c890d...	1002	101	"Withdrawal"
3	ObjectId('6a8966e0c4c890d...	1003	102	"Deposit"
4	ObjectId('6a8966e0c4c890d...	1004	102	"Withdrawal"
5	ObjectId('6a8966e0c4c890d...	1005	103	"Deposit"
6	ObjectId('6a8966e0c4c890d...	1006	103	"Withdrawal"
7	ObjectId('6a8966e0c4c890d...	1007	104	"Deposit"
8	ObjectId('6a8966e0c4c890d...	1008	104	"Withdrawal"

QUERY 2: FIND ACTIVE CUSTOMERS

Filter:

```
{
  "status": "Active"
}
```

Cluster0

customers

transactions

+

Cluster0 > bank_DB > customers

View monitoring

Visualize Your Data

Documents 8

Aggregations

Schema

Indexes 1

Validation

🕒

{ city: "Hyderabad" }

Generate query ⚡

Explain

Reset

Find

Options ▶

➕ ADD DATA ▼

📄 BULK ▼

🔗 EXPORT CODE ⓘ

25 ▼

1 - 3 of 3

↺

↻

↷

⋮

{ }

🏠

🏠 customers

	_id ObjectId	customer_id Int32	name String	city String	
1	ObjectId('6a8964e3c4c890d...	101	"Ravi Kumar"	"Hyderabad"	✎ 📄 🗑
2	ObjectId('6a8965bfc4c890d...	103	"Arjun Reddy"	"Hyderabad"	✎ 📄 🗑
3	ObjectId('6a8965bfc4c890d...	106	"Anjali Singh"	"Hyderabad"	✎ 📄 🗑

◀ ▶ 🔍

▶ 🔍 ▶

=====

QUERY 3: FIND CUSTOMERS WITH BALANCE GREATER THAN 100000

=====

Filter:

```
{
  "balance": {
    "$gt": 100000
  }
}
```

The screenshot shows the MongoDB Atlas web interface. At the top, the project is 'Project 0'. The database is 'bank_DB' and the collection is 'customers'. The query filter is entered as: `{ "balance": { "$gt": 100000 } }`. Below the query editor, there are buttons for 'Generate query', 'Explain', 'Reset', 'Find', and 'Options'. The 'Find' button is highlighted. Below the query editor, there are buttons for 'ADD DATA', 'BULK', and 'EXPORT CODE'. The table below shows the results of the query, displaying 4 records. The table has columns: _id, customer_id, name, and city. The records are: 1. Priya Sharma (Vijayawada), 2. Sneha Rao (Warangal), 3. Anjali Singh (Hyderabad), and 4. Meena Devi (Chennai).

	_id ObjectId	customer_id Int32	name String	city String	
1	ObjectId('6a8965bfc4c890d...	102	"Priya Sharma"	"Vijayawada"	
2	ObjectId('6a8965bfc4c890d...	104	"Sneha Rao"	"Warangal"	
3	ObjectId('6a8965bfc4c890d...	106	"Anjali Singh"	"Hyderabad"	
4	ObjectId('6a8965bfc4c890d...	108	"Meena Devi"	"Chennai"	

QUERY 4: FIND HYDERABAD CUSTOMERS

Filter:

```
{
  "city": "Hyderabad"
}
```

The screenshot displays the MongoDB Atlas web interface. At the top, the breadcrumb navigation shows 'Cluster0 > bank_DB > customers'. Below this, the 'Documents' tab is selected, showing 8 documents. A query filter is entered in the search bar: `{ "city": "Hyderabad" }`. To the right of the query bar are buttons for 'Generate query', 'Explain', 'Reset', 'Find', and 'Options'. Below the query bar, there are buttons for 'ADD DATA', 'BULK', 'EXPORT CODE', and a help icon. The results section shows a table with 3 documents. The table has columns: '_id ObjectId', 'customer_id Int32', 'name String', and 'city String'. The data rows are: 1. Ravi Kumar (customer_id 101), 2. Arjun Reddy (customer_id 103), and 3. Anjali Singh (customer_id 106). All three customers are from Hyderabad. At the bottom, there is a pagination bar showing '1 - 3 of 3' and navigation controls.

	_id ObjectId	customer_id Int32	name String	city String	
1	ObjectId('6a8964e3c4c890d...	101	"Ravi Kumar"	"Hyderabad"	   
2	ObjectId('6a8965bfc4c890d...	103	"Arjun Reddy"	"Hyderabad"	   
3	ObjectId('6a8965bfc4c890d...	106	"Anjali Singh"	"Hyderabad"	   

QUERY 5: FIND SAVINGS ACCOUNT CUSTOMERS

Filter:

```
{
  "account_type": "Savings"
}
```

Cluster0 > bank_DB > customers

[View monitoring](#)

[Visualize Your Data](#)

Documents 8

[Aggregations](#)

[Schema](#)

[Indexes](#) 1

[Validation](#)

```
{
  "account_type": "Savings"
}
```

[Generate query](#)

[Explain](#)

[Reset](#)

[Find](#)

[Options](#)

[+ ADD DATA](#)

[BULK](#)

[EXPORT CODE](#)

25

1 - 5 of 5

[↺](#) [↻](#) [↷](#)

[⌵](#)

[☰](#)

[⌄](#)

[⌚](#)

[🏠](#)

[customers](#)

	_id ObjectId	customer_id Int32	name String	city String	
1	ObjectId('6a8964e3c4c890d...	101	"Ravi Kumar"	"Hyderabad"	✎ 📄 🗑️
2	ObjectId('6a8965bfc4c890d...	103	"Arjun Reddy"	"Hyderabad"	✎ 📄 🗑️
3	ObjectId('6a8965bfc4c890d...	104	"Sneha Rao"	"Warangal"	✎ 📄 🗑️
4	ObjectId('6a8965bfc4c890d...	106	"Anjali Singh"	"Hyderabad"	✎ 📄 🗑️
5	ObjectId('6a8965bfc4c890d...	108	"Meena Devi"	"Chennai"	✎ 📄 🗑️

QUERY 6: FIND TRANSACTIONS GREATER THAN 20000

Open:

transactions

|

+---- Documents

Filter:

```
{
  "amount": {
    "$gt": 20000
  }
}
```

Cluster0 > bank_DB > transactions

[View monitoring](#)

[Visualize Your Data](#)

Documents 15

[Aggregations](#)

[Schema](#)

[Indexes](#) 1

[Validation](#)



```
{
  "amount": {
    "$gt": 20000
  }
}
```

[Generate query](#) ✦

[Explain](#)

[Reset](#)

[Find](#)

[Options](#) ▶

[ADD DATA](#) ▼

[BULK](#) ▼

[EXPORT CODE](#)



25 ▼

1 - 9 of 9



🏠 transactions

	_id ObjectId	transaction_id Int32	customer_id Int32	type String	
1	ObjectId('6a8966e0c4c890d...	1001	101	"Deposit"	
2	ObjectId('6a8966e0c4c890d...	1003	102	"Deposit"	
3	ObjectId('6a8966e0c4c890d...	1005	103	"Deposit"	
4	ObjectId('6a8966e0c4c890d...	1007	104	"Deposit"	
5	ObjectId('6a8966e0c4c890d...	1008	104	"Withdrawal"	

=====

QUERY 7: FIND DEPOSITS

=====

Filter:

```
{
  "type": "Deposit"
}
```

Cluster0 > bank_DB > transactions

View monitoring

Visualize Your Data

Documents 15

Aggregations

Schema

Indexes 1

Validation

🕒

{
 "type": "Deposit"
}

Generate query ⚡

Explain

Reset

Find

Options ▶

➕ ADD DATA ▾

📄 BULK ▾

🔗 EXPORT CODE ⓘ

25 ▾

1 - 8 of 8

↺

↻

↷

⌵

☰

|

{ }

🏠

🏠 transactions

	_id ObjectId	transaction_id Int32	customer_id Int32	type String	
1	ObjectId('6a8966e0c4c890d...	1001	101	"Deposit"	✎ 📄 🗑
2	ObjectId('6a8966e0c4c890d...	1003	102	"Deposit"	✎ 📄 🗑
3	ObjectId('6a8966e0c4c890d...	1005	103	"Deposit"	✎ 📄 🗑
4	ObjectId('6a8966e0c4c890d...	1007	104	"Deposit"	✎ 📄 🗑
5	ObjectId('6a8966e0c4c890d...	1009	105	"Deposit"	✎ 📄 🗑
6	ObjectId('6a8966e0c4c890d...	1010	106	"Deposit"	✎ 📄 🗑

QUERY 8: FIND ATM TRANSACTIONS

Filter:

```
{
  "mode": "ATM"
}
```

Cluster0

customers

transactions

+

Cluster0 > bank_DB > transactions

View monitoring
Visualize Your Data

Documents 15
Aggregations
Schema
Indexes 1
Validation

⌚
{
"mode": "ATM"
}

Generate query
Explain
Reset
Find
Options

+ ADD DATA
BULK
EXPORT CODE

25
1 - 7 of 7

transactions

	_id ObjectId	transaction_id Int32	customer_id Int32	mode String	
1	ObjectId('6a8966e0c4c890d...	1002	101	"ATM"	
2	ObjectId('6a8966e0c4c890d...	1004	102	"ATM"	
3	ObjectId('6a8966e0c4c890d...	1006	103	"ATM"	
4	ObjectId('6a8966e0c4c890d...	1008	104	"ATM"	
5	ObjectId('6a8966e0c4c890d...	1011	106	"ATM"	
6	ObjectId('6a8966e0c4c890d...	1013	107	"ATM"	

QUERY 9: FIND DEPOSITS GREATER THAN 50000

Filter:

```
{
  "type": "Deposit",
  "amount": {
    "$gt": 50000
  }
}
```

Cluster0 > bank_DB > transactions

[View monitoring](#)

[Visualize Your Data](#)

Documents 15

Aggregations

Schema

Indexes 1

Validation



```
{
  "type": "Deposit",
  "amount": {
    "$gt": 50000
  }
}
```

[Generate query](#)

[Explain](#)

[Reset](#)

[Find](#)

[Options](#)

[ADD DATA](#)

[BULK](#)

[EXPORT CODE](#)



25

1 - 3 of 3



transactions

	_id ObjectId	transaction_id Int32	customer_id Int32	type String	
1	ObjectId('6a8966e0c4c890d...	1007	104	"Deposit"	
2	ObjectId('6a8966e0c4c890d...	1010	106	"Deposit"	
3	ObjectId('6a8966e0c4c890d...	1014	108	"Deposit"	



=====

QUERY 10: FIND HYDERABAD CUSTOMERS WITH HIGH BALANCE

=====

Filter:

```
{
  "city": "Hyderabad",
  "balance": {
    "$gt": 100000
  }
}
```

The screenshot shows the MongoDB Compass interface. At the top, there's a breadcrumb navigation: Cluster0 > bank_DB > customers. Below this, there are tabs for Documents (8), Aggregations, Schema, Indexes (1), and Validation. The Documents tab is active. In the center, a query filter is entered: `{ "city": "Hyderabad", "balance": { "$gt": 100000 } }`. To the right of the filter are buttons: Generate query, Explain, Reset, Find, and Options. Below the filter, there's a toolbar with buttons: ADD DATA, BULK, EXPORT CODE, and a status bar showing 25 results, 1 of 1. At the bottom, a table displays the data for the 'customers' collection. The table has columns: customer_id (Int32), name (String), city (String), and account_type (String). The first row shows a customer with ID 106, name 'Anjali Singh', city 'Hyderabad', and account type 'Savings'.

	customer_id Int32	name String	city String	account_type String
1	106	"Anjali Singh"	"Hyderabad"	"Savings"

QUERY 11: FIND CUSTOMERS IN MULTIPLE CITIES

Filter:

```
{
  "city": {
    "$in": [
      "Hyderabad",
      "Vijayawada",
      "Warangal"
    ]
  }
}
```

The screenshot shows the MongoDB Compass interface. At the top, the breadcrumb navigation indicates the path: Cluster0 > bank_DB > customers. Below this, the 'Documents' tab is selected, showing 8 documents. The query filter is entered in the main editor as follows:

```
{
  "city": {
    "$in": [
      "Hyderabad",
      "Vijayawada",
      "Warangal"
    ]
  }
}
```

Below the query editor, there are buttons for 'ADD DATA', 'BULK', 'EXPORT CODE', and a status bar showing '25' and '1 - 5 of 5'. The results table below shows the following data:

	_id ObjectId	customer_id Int32	name String	city String
1	ObjectId('6a8964e3c4c890d...	101	"Ravi Kumar"	"Hyderabad"
2	ObjectId('6a8965bfc4c890d...	102	"Priya Sharma"	"Vijayawada"
3	ObjectId('6a8965bfc4c890d...	103	"Arjun Reddy"	"Hyderabad"

QUERY 12: SORT CUSTOMERS BY BALANCE

Use:

Aggregations

Add Stage:

\$sort

Enter:

```
{
  "balance": -1
}
```

The screenshot shows the MongoDB Atlas Aggregations editor interface. The breadcrumb path is Cluster0 > bank_DB > customers. The 'Aggregations' tab is selected, showing a single stage named '\$sort'. The query for this stage is:

```
{ "balance": -1 }
```

. The 'Preview' button is active, and the output preview shows a sample of 8 documents. The first document in the preview is:

```
{ "_id": ObjectId('6a8965bfc4c890d41f9f6bce'), "customer_id": 108, "name": "Meena Devi", "city": "Chennai", "account_type": "Savings", "balance": 320000, "status": "Active" }
```

.

Cluster0 > bank_DB > customers

Documents 8 Aggregations Schema Indexes 1 Validation

\$sort

Generate aggregation Explain Run Options

Untitled - modified SAVE CREATE NEW EXPORT CODE PREVIEW STAGES TEXT WIZARD

Stage1 \$sort

```
1 {
2   balance: -1
3 }
```

Output preview after \$sort stage (Sample of 8 documents)

```
_id: ObjectId('6a8965bfc4c890d41f9f6bce')
customer_id: 108
name: "Meena Devi"
city: "Chennai"
account_type: "Savings"
balance: 320000
status: "Active"
```

QUERY 13: CALCULATE TOTAL TRANSACTION AMOUNT

Go to:

transactions

|

+---- Aggregations

Add Stage:

\$group

Enter:

```
{
  "_id": null,
  "total_amount": {
    "sum": "$balance"
  }
}
```

The screenshot shows the MongoDB Atlas Aggregations editor interface. At the top, the breadcrumb navigation indicates the path: Cluster0 > bank_DB > customers. There are buttons for 'View monitoring' and 'Visualize Your Data'. Below the breadcrumb, tabs for 'Documents' (8), 'Aggregations' (selected), 'Schema', 'Indexes' (1), and 'Validation' are visible. The main editor area shows a single aggregation stage named '\$group'. To the right of the stage name are buttons for 'Generate aggregation', 'Explain', 'Run', and 'Options'. Below the stage name, there is a toolbar with 'SAVE', 'CREATE NEW', 'EXPORT CODE', 'PREVIEW' (selected), 'STAGES', 'TEXT', 'WIZARD', and a settings icon. The 'PREVIEW' tab is active, showing the output preview after the '\$group' stage. The output is a sample of 1 document: { "_id": null, "total_amount": 1120000 }.

Cluster0 > bank_DB > customers

Documents 8 Aggregations Schema Indexes 1 Validation

\$group

Generate aggregation Explain Run Options

Untitled - modified SAVE CREATE NEW EXPORT CODE PREVIEW STAGES TEXT WIZARD

Stage 1 \$group

```
1 {
2   "_id": null,
3   "total_amount": {
4     "sum": "$balance"
5   }
6 }
```

Output preview after \$group stage (Sample of 1 document)

```
_id: null
total_amount: 1120000
```

=====

QUERY 14: CALCULATE AVERAGE TRANSACTION AMOUNT

=====

Aggregation:

```
{
  "_id": null,
  "average_amount": {
    "avg": "balance"
  }
}
```

The screenshot displays the MongoDB Compass web interface for a cluster named 'Cluster0', specifically within the 'bank_DB' database and 'customers' collection. The 'Aggregations' tab is active, showing a single stage named '\$group'. The aggregation pipeline is defined as follows:

```
1 {
2   "_id": null,
3   "average_amount": {
4     "$avg": "$balance"
5   }
6 }
7 |
```

The interface includes a top navigation bar with 'Documents' (8), 'Aggregations', 'Schema', 'Indexes' (1), and 'Validation'. Below the aggregation editor, there are buttons for 'Generate aggregation', 'Explain', 'Run', and 'Options'. The 'Preview' tab is selected, showing the output of the aggregation stage. The output is a single document with the following structure:

```
_id: null
average_amount : 140000
```

The interface also features a 'View monitoring' button, a 'Visualize Your Data' button, and a 'WIZARD' button. The 'Export Code' button is also visible.

QUERY 15: TOTAL TRANSACTION AMOUNT BY TYPE

Aggregation:

```
[
  {
    "$group": {
      "_id": "$type",
      "total_amount": {
        "sum": "amount"
      }
    }
  }
]
```

The screenshot displays the MongoDB Compass interface for a database named 'bank_DB' and a collection named 'transactions'. The 'Aggregations' tab is active, showing a pipeline with one stage: '\$group' with '_id' as '\$type' and 'total_amount' as a sum of 'amount'. The results section shows two documents:

_id	total_amount
Deposit	485000
Withdrawal	117000

The interface includes navigation tabs for Documents (15), Aggregations, Schema, Indexes (1), and Validation. It also features buttons for 'View monitoring', 'Visualize Your Data', 'Explain', 'Run', and 'Options'.

QUERY 16: NUMBER OF TRANSACTIONS PER CUSTOMER

Aggregation:

```
[
  {
    "$group": {
      "_id": "$customer_id",
      "transaction_count": {
        "$sum": 1
      }
    }
  },
]
```

Untitled - modified



+ CREATE NEW



PREVIEW



Stage 1 \$group

```
1 {
2   "_id": "$customer_id",
3   "transaction_count": {
4     "$sum": 1
5   }
6 }
```

Output preview after \$group stage (Sample of 8 documents)

```
_id: 102
transaction_count : 2
```

```
{
  "$sort": {
    "transaction_count": -1
  }
}
```

ALL RESULTS



Showing 1 - 15 count results



```
_id: ObjectId('6a8966e0c4c890d41f9f6bd5')
transaction_id: 1005
customer_id: 103
type: "Deposit"
amount: 30000
mode: "UPI"
date: "2026-08-03"
status: "Success"
```

```
_id: ObjectId('6a8966e0c4c890d41f9f6bd2')
transaction_id: 1002
customer_id: 101
type: "Withdrawal"
amount: 10000
mode: "ATM"
```

QUERY 17: CUSTOMER + TRANSACTION JOIN

Add Stage:

\$lookup

Enter:

```
{
  "from": "customers",
  "localField": "customer_id",
  "foreignField": "customer_id",
  "as": "customer_details"
}
```

The screenshot shows the MongoDB Atlas aggregation pipeline editor. The pipeline is named "Stage 1: \$lookup" and is enabled. The stage is configured with the following options:

- from: customers
- localField: customer_id
- foreignField: customer_id
- as: customer_details

The stage input shows a sample of 10 documents. The stage output shows a sample of 10 documents, where each document now includes the "customer_details" field, which is an array of documents from the "customers" collection.

STAGE INPUT: Sample of 10 documents

```
{ "_id": "ObjectId('6a8966...)", "transaction_id": 1005, "customer_id": 103, "type": "Deposit", "amount": 30000, "mode": "UPI", "date": "2026-08-03", "status": "Success" }
```

STAGE OUTPUT: Sample of 10 documents

```
{ "_id": "ObjectId('6a8966...)", "transaction_id": 1005, "customer_id": 103, "type": "Deposit", "amount": 30000, "mode": "UPI", "date": "2026-08-03", "status": "Success", "customer_details": [ { "from": "customers", "localField": "customer_id", "foreignField": "customer_id", "as": "customer_details" } ] }
```

Stage 1: \$lookup

ENABLED

ADD STAGE

STAGE INPUT

OPTIONS

Sample of 10 documents

_id: ObjectId('6a8966...

transaction_id: 1005

customer_id: 103

type: "Deposit"

amount: 30000

mode: "UPI"

date: "2026-08-03"

status: "Success"

_id: ObjectId('6a8966...

transaction_id: 1002

customer_id: 101

type: "Withdrawal"

amount: 10000

mode: "ATM"

date: "2026-08-02"

status: "Success"

_id: ObjectId('6a8966...

transaction_id: 1003

customer_id: 102

type: "Deposit"

amount: 50000

mode: "NEFT"

date: "2026-08-02"

status: "Success"

\$lookup

Open docs

```
1 {
2   "from": "customers",
3   "localField": "customer_id",
4   "foreignField": "customer_id",
5   "as": "customer_details"
6 }
7
```

STAGE OUTPUT

OPTIONS

Sample of 10 documents

_id: ObjectId('6a8966...

transaction_id: 1005

customer_id: 103

type: "Deposit"

amount: 30000

mode: "UPI"

date: "2026-08-03"

status: "Success"

customer_details: Arr...

_id: ObjectId('6a8966...

transaction_id: 1002

customer_id: 101

type: "Withdrawal"

amount: 10000

mode: "ATM"

date: "2026-08-02"

status: "Success"

customer_details: Arr...

_id: ObjectId('6a8966...

transaction_id: 1003

customer_id: 102

type: "Deposit"

amount: 50000

mode: "NEFT"

date: "2026-08-02"

status: "Success"

customer_details: Arr...

Go to previous stage Ctrl + Shift + 9

Stage 2: \$unwind

ENABLED

ADD STAGE

STAGE INPUT

OPTIONS

Sample of 10 documents

_id: ObjectId('6a8966...

transaction_id: 1005

customer_id: 103

type: "Deposit"

amount: 30000

mode: "UPI"

date: "2026-08-03"

status: "Success"

customer_details: Arr...

_id: ObjectId('6a8966...

transaction_id: 1002

customer_id: 101

type: "Withdrawal"

amount: 10000

mode: "ATM"

date: "2026-08-02"

status: "Success"

customer_details: Arr...

_id: ObjectId('6a8966...

transaction_id: 1003

customer_id: 102

type: "Deposit"

amount: 50000

mode: "NEFT"

date: "2026-08-02"

status: "Success"

customer_details: Arr...

\$unwind

Open docs

```
1 "$customer_details"
```

STAGE OUTPUT

OPTIONS

Sample of 10 documents

_id: ObjectId('6a8966...

transaction_id: 1005

customer_id: 103

type: "Deposit"

amount: 30000

mode: "UPI"

date: "2026-08-03"

status: "Success"

customer_details: Obj...

_id: ObjectId('6a8966...

transaction_id: 1002

customer_id: 101

type: "Withdrawal"

amount: 10000

mode: "ATM"

date: "2026-08-02"

status: "Success"

customer_details: Obj...

_id: ObjectId('6a8966...

transaction_id: 1003

customer_id: 102

type: "Deposit"

amount: 50000

mode: "NEFT"

date: "2026-08-02"

status: "Success"

customer_details: Obj...

QUERY 18: DISPLAY CUSTOMER NAME AND TRANSACTION AMOUNT

Use:

\$lookup

+

\$unwind

+

\$project

Pipeline:

```
[
  {
    "$lookup": {
      "from": "customers",
      "localField": "customer_id",
      "foreignField": "customer_id",
      "as": "customer"
    }
  },
  {
    "$unwind": "$customer"
  },
  {
    "$project": {
      "_id": 0,
      "transaction_id": 1,
      "customer_name": "$customer.name",
      "city": "$customer.city",
      "type": 1,
      "amount": 1,
      "mode": 1
    }
  }
]
```

Cluster0 > bank_DB > transactions

 View monitoring

 Visualize Your Data

Documents 15

Aggregations

Schema

Indexes 1

Validation

 \$lookup \$unwind \$project



Explain

Run

Options ▶

Untitled - modified



+ CREATE NEW



 PREVIEW



▼ Stage 3 \$project



```
1 {
2   "_id": 0,
3   "transaction_id": 1,
4   "customer_name": "$customer_details.name",
5   "city": "$customer_details.city",
6   "type": 1,
7   "amount": 1,
8   "mode": 1
9 }
```

Output preview after \$project stage (Sample of 10 documents)

```
transaction_id : 1005
type : "Deposit"
amount : 30000
mode : "UPI"
customer_name : "Arjun Reddy"
city : "Hyderabad"
```

QUERY 19: TOTAL TRANSACTION AMOUNT FOR EACH CUSTOMER

Pipeline:

```
[
  {
    "$group": {
      "_id": "$customer_id",
      "total_transaction": {
        "$sum": "$amount"
      }
    },
  },
  {
    "$lookup": {
      "from": "customers",
      "localField": "_id",
      "foreignField": "customer_id",
      "as": "customer"
    }
  },
  {
    "$unwind": "$customer"
  },
  {
    "$project": {
      "_id": 0,
      "customer_id": "$_id",
      "customer_name": "$customer.name",
      "total_transaction": 1
    }
  },
  {
    "$sort": {
      "total_transaction": -1
    }
  }
]
```

Stage 5: \$sort

ENABLED

ADD STAGE

STAGE OUTPUT

Sample of 8 documents

total_transaction	customer_id	customer_name
180	108	Meena Devi
120	104	Sneha Rao
950	106	Anjali Singh
650	102	Priya Sharma
520	107	Rahul Verma

=====

QUERY 20: COMPLEX BANK TRANSACTION REPORT

=====

PIPELINE:

```
[
  {
    "$match": {
      "status": "Active",
      "balance": {
        "$gt": 80000
      }
    }
  },
  {
    "$lookup": {
      "from": "transactions",
      "localField": "customer_id",
      "foreignField": "customer_id",
      "as": "transactions"
    }
  },
  {
    "$unwind": "$transactions"
  },
  {
    "$group": {
      "_id": "$customer_id",
      "customer_name": {
        "$first": "$name"
      },
      "city": {
        "$first": "$city"
      },
      "balance": {
        "$first": "$balance"
      },
      "total_transactions": {
        "$sum": "$transactions.amount"
      },
      "transaction_count": {
        "$sum": 1
      }
    }
  },
]
```

```

{
  "$sort": {
    "total_transactions": -1
  }
}
]

```

The screenshot shows the MongoDB Atlas pipeline editor interface. A dropdown menu on the left lists stages: Stage 1: \$match, Stage 2: \$lookup, Stage 3: \$unwind, Stage 4: \$group, and Stage 5: \$sort (which is selected and checked). The main editor area shows Stage 5: \$sort is enabled. A dropdown menu for the stage shows the operation \$sort, and a link 'Open docs' is available. The stage output is displayed as a sample of 6 documents, sorted by total_transactions in descending order.

STAGE OUTPUT
Sample of 6 documents

_id	customer_name	city	balance	total_transactions	transaction_count
108	Meena Devi	Chennai	320000	1	2
104	Sneha Rao	Warangal	210000	1	2
106	Anjali Singh	Hyderabad	175000	9	2
102	Priya Sharma	Vijayawada	125000	6	2
108	Meena Devi	Chennai	320000	1	2
104	Sneha Rao	Warangal	210000	1	2